FE515: Linear Regression Code Walkthrough

Sean O'Leary

Goal for Today

Can we use a house's characteristics to estimate its price?

We will start with square footage, add neighborhood information, and then let R compare which variables to keep. Along the way, we will look at what makes one model fit better than another.

Keep these two CSV files in the same folder as this R Markdown file:

- `FE515_L7_house_prices.csv`
- `FE515_L7_house_prices_multiple.csv`

All of today's modeling functions are included with R. No additional modeling packages are needed.

Simple Linear Regression

The model

Simple linear regression uses one explanatory variable to model a response:

$$y_i = \alpha + \beta x_i + \epsilon_i.$$

- y_i: what we want to explain or predict.
- x_i: the variable we use to explain it.
- α: the intercept.
- β: the slope.
- ϵ_i: the part not explained by the model.

For our housing example:

$$\text{Price}_i = \alpha + \beta \cdot \text{SquareFootage}_i + \epsilon_i.$$

We expect larger houses to cost more, but two houses of the same size can have different prices. The error term allows for that difference.

Load and inspect the data

```
houses <- read.csv("FE515_L7_house_prices.csv", check.names = FALSE)

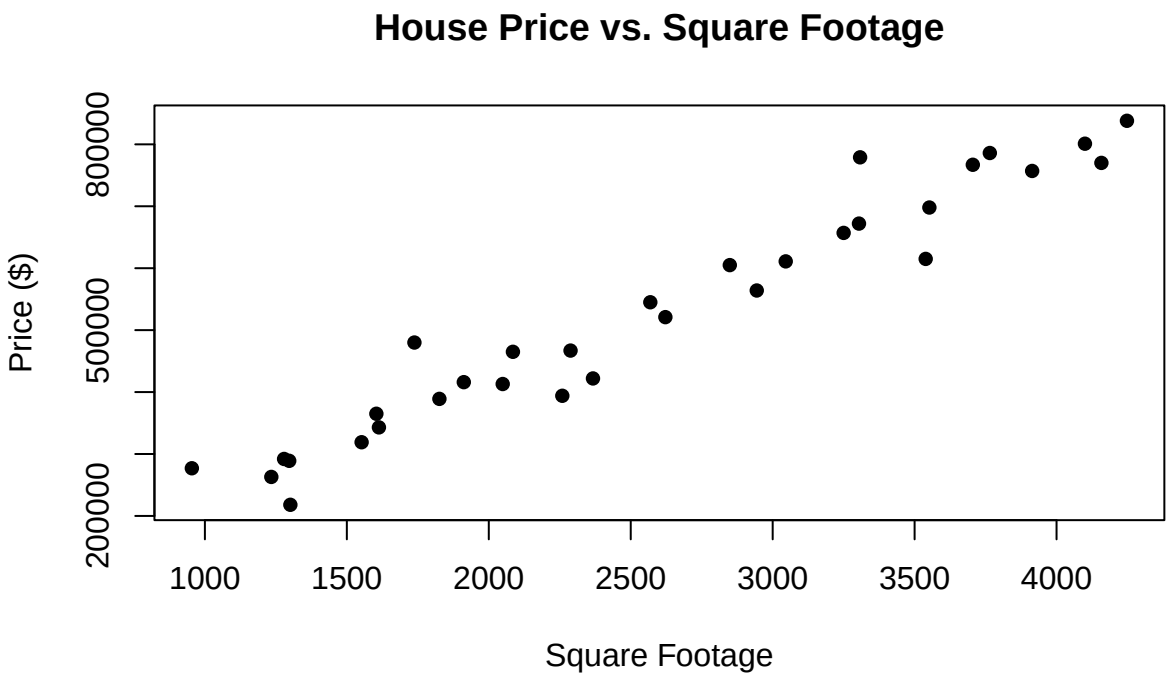
# Clean the column names so they are easier to type
names(houses) <- trimws(names(houses))
names(houses) <- make.names(names(houses))

head(houses)
```

```
##   SquareFootage  Price
## 1          2259 394000
## 2          4248 838000
## 3          2849 605000
## 4          3046 611000
## 5          3765 786000
## 6          1613 343000
```

Before fitting a model, plot the observations.

```
plot(
  houses$SquareFootage,
  houses$Price,
  xlab = "Square Footage",
  ylab = "Price ($)",
  main = "House Price vs. Square Footage",
  pch = 16
)
```



How does R find the line?

For any proposed intercept and slope, the predicted price is

$$\hat{y}_i = \alpha + \beta x_i.$$

The residual is the actual price minus the predicted price:

$$e_i = y_i - \hat{y}_i.$$

A positive residual means the house sold for more than predicted. A negative residual means it sold for less.

Least squares chooses the intercept and slope that minimize the total squared residuals:

$$\min_{\alpha,\beta} \sum_{i=1}^{n} (y_i - \alpha - \beta x_i)^2.$$

Squaring prevents positive and negative errors from canceling and gives larger errors more weight. We do not need to solve this by hand: `lm()` does it for us.

```
house_model <- lm(Price ~ SquareFootage, data = houses)
summary(house_model)
```

```
##
## Call:
## lm(formula = Price ~ SquareFootage, data = houses)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -87200 -16946  -2111  17554 119045
##
## Coefficients:
##               Estimate Std. Error t value            Pr(>|t|)
## (Intercept)   54986.13   22413.50    2.45                0.02 *
## SquareFootage   182.88       8.15   22.44 <0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 45100 on 30 degrees of freedom
## Multiple R-squared:  0.944,  Adjusted R-squared:  0.942
## F-statistic:  503 on 1 and 30 DF,  p-value: <0.0000000000000002
```

In an R formula, the response goes to the left of `~` and the explanatory variables go to the right. R includes an intercept by default.

The fitted equation is approximately

$$\widehat{\text{Price}} = 54{,}986 + 182.88 \cdot \text{SquareFootage}.$$

An additional square foot is associated with about $183 more in predicted price. The intercept is where the fitted line crosses the vertical axis; a zero-square-foot house is outside our data, so it has little practical meaning here.

See the residuals

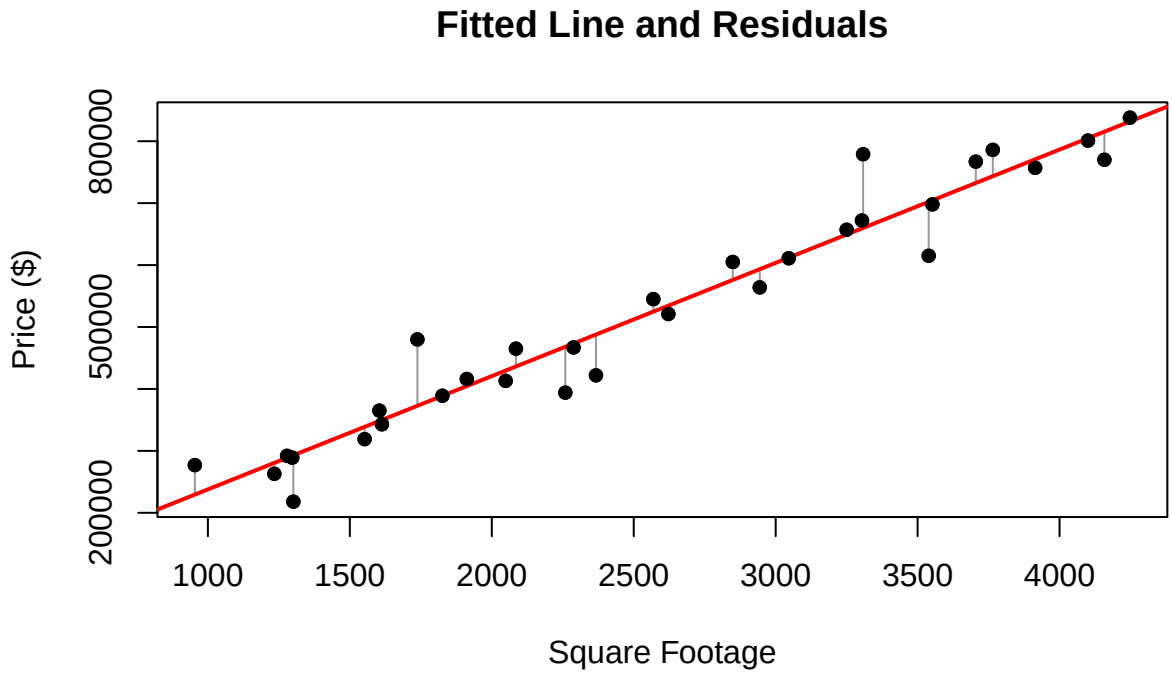
The vertical gray segments show the difference between each actual price and its fitted price. These are the errors that least squares minimizes after squaring them.

```r
plot(
  houses$SquareFootage,
  houses$Price,
  xlab = "Square Footage",
  ylab = "Price ($)",
  main = "Fitted Line and Residuals",
  pch = 16
)

segments(
  x0 = houses$SquareFootage,
  y0 = houses$Price,
```

```
  x1 = houses$SquareFootage,
  y1 = fitted(house_model),
  col = "gray60"
)

abline(house_model, col = "red", lwd = 2)
points(houses$SquareFootage, houses$Price, pch = 16)
```



We can also inspect the fitted prices and residuals directly.

```
head(data.frame(
  ActualPrice = houses$Price,
  PredictedPrice = fitted(house_model),
  Residual = resid(house_model)
))
```

```
##   ActualPrice PredictedPrice Residual
## 1      394000         468113   -74113
## 2      838000         831862     6138
## 3      605000         576013    28987
## 4      611000         612040    -1040
## 5      786000         743531    42469
## 6      343000         349972    -6972
```

```r
# Total squared residuals
sum(resid(house_model)^2)
```

```
## [1] 61141541247
```

What does R-squared mean?

Suppose we ignored square footage and predicted the average house price for every house. The total squared error from that baseline is

$$SST = \sum_{i=1}^{n} (y_i - \bar{y})^2.$$

Our regression model instead has squared error

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2.$$

R-squared compares the two:

$$R^2 = 1 - \frac{SSE}{SST}.$$

For ordinary least squares with an intercept, $R^2 = 0$ means the model does no better in this sample than predicting the mean. $R^2 = 1$ means every fitted price matches its actual price.

```r
SST <- sum((houses$Price - mean(houses$Price))^2)
SSE <- sum(resid(house_model)^2)

R_squared <- 1 - SSE / SST
R_squared
```

```
## [1] 0.9438
```

```r
# Check against R's model summary
summary(house_model)$r.squared
```

```
## [1] 0.9438
```

Here, R^2 is about 0.944: the fitted relationship with square footage explains about 94.4% of the variation in prices in this sample. This is a measure of fit on the observed data, not a guarantee about new houses.

Reading the output

Focus on these parts of `summary()`:

- **Estimate:** the fitted intercept and slopes.
- **Pr(>|t|):** the p-value for testing whether a coefficient is zero.
- **Multiple R-squared:** the proportion of sample variation explained by the fitted model.

A small p-value means an estimate this far from zero, relative to its uncertainty, would be unusual if the true coefficient were zero, assuming the model's testing assumptions hold. It is evidence of an association, not proof of causation.

Predict a price

What price does the model predict for a 2,500-square-foot house?

```
new_house <- data.frame(SquareFootage = 2500)
predict(house_model, newdata = new_house)
```

```
##      1
## 512187
```

The prediction is about $512,187. `predict()` substitutes the new square footage into the fitted equation.

Multiple Regression

The model

House prices also depend on characteristics beyond size. Multiple regression lets us include several explanatory variables:

$$y_i = \alpha + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \epsilon_i.$$

We will use square footage, a neighborhood crime index, and distance to the city. R still chooses the coefficients by minimizing squared residuals.

Load the second dataset

```r
houses_multi <- read.csv(
  "FE515_L7_house_prices_multiple.csv",
  check.names = FALSE
)

names(houses_multi) <- trimws(names(houses_multi))
names(houses_multi) <- make.names(names(houses_multi))

head(houses_multi)
```

```
##    Price SquareFootage CrimeRate DistanceToCity
## 1 519000          2873       3.1           22.1
## 2 236000          1651       5.9           21.7
## 3 230000          1461       6.5           26.6
## 4 445000          2031       5.8            3.1
## 5 276000           932       1.2           10.4
## 6 708000          4060       5.2           25.7
```

The variables are:

- `Price`: sale price in dollars.
- `SquareFootage`: house size in square feet.
- `CrimeRate`: neighborhood crime index.
- `DistanceToCity`: distance to the city in miles.

Fit the model

```r
multi_model <- lm(
  Price ~ SquareFootage + CrimeRate + DistanceToCity,
  data = houses_multi
)

summary(multi_model)
```

```
##
## Call:
## lm(formula = Price ~ SquareFootage + CrimeRate + DistanceToCity,
##     data = houses_multi)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -71071 -21415  -2676  28029  54107
##
## Coefficients:
```

```
##                 Estimate Std. Error t value             Pr(>|t|)
## (Intercept)    135813.9    18525.0    7.33     0.00000000563447 ***
## SquareFootage     186.6        4.5   41.47 < 0.0000000000000002 ***
## CrimeRate       -8127.5     2062.0   -3.94              0.00031 ***
## DistanceToCity  -5659.9      535.5  -10.57     0.00000000000028 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 29600 on 41 degrees of freedom
## Multiple R-squared:  0.978,  Adjusted R-squared:  0.977
## F-statistic:  620 on 3 and 41 DF,  p-value: <0.0000000000000002
```

```r
coef(multi_model)
```

```
##    (Intercept)  SquareFootage      CrimeRate DistanceToCity
##       135813.9          186.6        -8127.5        -5659.9
```

The fitted equation is approximately

$$\begin{aligned}\widehat{\text{Price}} = 135{,}814 &+ 186.61 \cdot \text{SquareFootage} \\ &- 8{,}127.53 \cdot \text{CrimeRate} \\ &- 5{,}659.93 \cdot \text{DistanceToCity}.\end{aligned}$$

Interpret a coefficient

Each slope describes a change in predicted price **holding the other model variables fixed**.

For example, if square footage increases by 100 and crime rate and distance stay unchanged:

$$\Delta\widehat{\text{Price}} = 186.61 \times 100 \approx \$18{,}661.$$

```r
100 * coef(multi_model)["SquareFootage"]
```

```
## SquareFootage
##         18661
```

Similarly, one additional mile from the city is associated with about \$5,660 less in predicted price, holding size and crime rate fixed.

Predict with several inputs

Consider a house with 2,500 square feet, a crime index of 3.5, and a distance of 8 miles.

```r
new_house_multi <- data.frame(
  SquareFootage = 2500,
  CrimeRate = 3.5,
  DistanceToCity = 8
)

predict(multi_model, newdata = new_house_multi)
```

```
##      1
## 528623
```

The predicted price is about $528,623.

Compare models on the same houses

The two CSV files contain different observations. To see what adding crime rate and distance contributes, fit a size-only model on the second dataset too.

```r
size_only_model <- lm(Price ~ SquareFootage, data = houses_multi)

data.frame(
  Model = c("Size only", "Size, crime, and distance"),
  R_squared = c(
    summary(size_only_model)$r.squared,
    summary(multi_model)$r.squared
  )
)
```

```
##                       Model R_squared
## 1                 Size only    0.9156
## 2 Size, crime, and distance    0.9784
```

Adding predictors cannot reduce ordinary in-sample R^2 when the models use the same observations and one contains the other. But even an irrelevant variable can improve fit slightly. Should we always keep it?

Stepwise Regression

Add a random dummy variable

We will give each house a random label of 0 or 1. It represents no house characteristic and was generated independently of the housing data.

```r
houses_multi$RandomDummy <- as.integer(runif(nrow(houses_multi)) > 0.5)

head(houses_multi)
```

```
##    Price SquareFootage CrimeRate DistanceToCity RandomDummy
## 1 519000          2873       3.1           22.1           0
## 2 236000          1651       5.9           21.7           1
## 3 230000          1461       6.5           26.6           1
## 4 445000          2031       5.8            3.1           0
## 5 276000           932       1.2           10.4           0
## 6 708000          4060       5.2           25.7           0
```

```r
table(houses_multi$RandomDummy)
```

```
##
##  0  1
## 26 19
```

`runif()` generates random numbers between 0 and 1. Values above 0.5 become 1; the rest become 0.

A dummy variable is a 0/1 indicator. Real dummy variables can represent useful categories, such as whether a house has a garage. This one is just a random label for our demonstration.

With the dummy included, the model has one extra term:

$$y_i = \alpha + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \beta_4 D_i + \epsilon_i.$$

The coefficient on D_i is the predicted price difference between labels 1 and 0, holding the other predictors fixed. We have no reason to expect a useful difference, but the sample may show one by chance.

Fit versus complexity

Stepwise regression compares nearby models by adding or removing a variable at a time. Here, `step()` uses AIC, which balances fit against the number of estimated parameters. **Lower AIC is preferred** when comparing models fitted to the same response observations.

An extra variable must improve fit enough to justify the added complexity. AIC selection does not use a fixed p-value cutoff.

There are three search directions:

- **Forward:** start with an intercept only and consider adding variables.
- **Backward:** start with all candidate variables and consider removing them.
- **Both:** allow additions and removals as the search proceeds.

Define the candidate models

```
null_model <- lm(Price ~ 1, data = houses_multi)

full_model <- lm(
  Price ~ SquareFootage + CrimeRate + DistanceToCity + RandomDummy,
  data = houses_multi
)

scope_list <- list(
  lower = ~ 1,
  upper = ~ SquareFootage + CrimeRate + DistanceToCity + RandomDummy
)

summary(full_model)
```

```
##
## Call:
## lm(formula = Price ~ SquareFootage + CrimeRate + DistanceToCity +
##     RandomDummy, data = houses_multi)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -64461 -21992  -5544  23420  58403
##
## Coefficients:
##                 Estimate Std. Error t value             Pr(>|t|)
## (Intercept)    140916.16   18895.17    7.46     0.00000000435566 ***
## SquareFootage     186.86       4.48   41.72 < 0.0000000000000002 ***
## CrimeRate       -7569.71    2101.35   -3.60              0.00086 ***
## DistanceToCity  -5871.82     560.40  -10.48     0.00000000000049 ***
## RandomDummy    -11704.14    9658.72   -1.21              0.23271
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 29500 on 40 degrees of freedom
## Multiple R-squared:  0.979,  Adjusted R-squared:  0.977
## F-statistic:  471 on 4 and 40 DF,  p-value: <0.0000000000000002
```

`Price ~ 1` predicts the average price for every house. The full model includes all four candidates. The scope tells R which variables it is allowed to consider.

Forward selection

```r
forward_model <- step(
  null_model,
  scope = scope_list,
  direction = "forward"
)
```

```
## Start:  AIC=1097
## Price ~ 1
##
##                   Df     Sum of Sq           RSS  AIC
## + SquareFootage    1 1530710732074  141077267926  988
## + DistanceToCity   1  102257296025 1569530703975 1096
## <none>                             1671788000000 1097
## + CrimeRate        1   12884340925 1658903659075 1099
## + RandomDummy      1    8735232794 1663052767206 1099
##
## Step:  AIC=988
## Price ~ SquareFootage
##
##                   Df   Sum of Sq          RSS AIC
## + DistanceToCity   1 91394385612  49682882313 943
## + CrimeRate        1  6860666566 134216601360 988
## <none>                           141077267926 988
## + RandomDummy      1  1756745608 139320522318 989
##
## Step:  AIC=943
## Price ~ SquareFootage + DistanceToCity
##
##                Df   Sum of Sq         RSS AIC
## + CrimeRate     1 13652871886 36030010428 931
## + RandomDummy   1  3653931993 46028950321 942
## <none>                        49682882313 943
##
## Step:  AIC=930.5
## Price ~ SquareFootage + DistanceToCity + CrimeRate
##
##                Df  Sum of Sq         RSS AIC
## <none>                       36030010428 931
## + RandomDummy   1 1275813521 34754196907 931
```

At each step, look at the AIC column. `<none>` means keeping the current model. R adds a variable if doing so lowers the score, then repeats the comparison.

Backward selection

```
backward_model <- step(
  full_model,
  scope = scope_list,
  direction = "backward"
)
```

```
## Start:  AIC=930.9
## Price ~ SquareFootage + CrimeRate + DistanceToCity + RandomDummy
##
##                   Df     Sum of Sq           RSS  AIC
## - RandomDummy      1    1275813521   36030010428  931
## <none>                               34754196907  931
## - CrimeRate        1   11274753414   46028950321  942
## - DistanceToCity   1   95387301625  130141498532  988
## - SquareFootage    1 1511979497700 1546733694607 1100
##
## Step:  AIC=930.5
## Price ~ SquareFootage + CrimeRate + DistanceToCity
##
##                   Df     Sum of Sq           RSS  AIC
## <none>                               36030010428  931
## - CrimeRate        1   13652871886   49682882313  943
## - DistanceToCity   1   98186590932  134216601360  988
## - SquareFootage    1 1511145948958 1547175959385 1098
```

This time, R begins with every candidate and checks whether removing a variable lowers AIC.

Both directions

```
both_model <- step(
  null_model,
  scope = scope_list,
  direction = "both"
)
```

```
## Start:  AIC=1097
## Price ~ 1
##
##                   Df     Sum of Sq           RSS  AIC
## + SquareFootage    1 1530710732074  141077267926  988
## + DistanceToCity   1  102257296025 1569530703975 1096
## <none>                             1671788000000 1097
```

```
## + CrimeRate       1   12884340925 1658903659075 1099
## + RandomDummy     1    8735232794 1663052767206 1099
##
## Step:  AIC=988
## Price ~ SquareFootage
##
##                   Df     Sum of Sq           RSS  AIC
## + DistanceToCity  1   91394385612   49682882313  943
## + CrimeRate       1    6860666566  134216601360  988
## <none>                             141077267926  988
## + RandomDummy     1    1756745608  139320522318  989
## - SquareFootage   1 1530710732074 1671788000000 1097
##
## Step:  AIC=943
## Price ~ SquareFootage + DistanceToCity
##
##                   Df     Sum of Sq           RSS  AIC
## + CrimeRate       1   13652871886   36030010428  931
## + RandomDummy     1    3653931993   46028950321  942
## <none>                             49682882313  943
## - DistanceToCity  1   91394385612  141077267926  988
## - SquareFootage   1 1519847821661 1569530703975 1096
##
## Step:  AIC=930.5
## Price ~ SquareFootage + DistanceToCity + CrimeRate
##
##                   Df     Sum of Sq           RSS  AIC
## <none>                             36030010428  931
## + RandomDummy     1    1275813521   34754196907  931
## - CrimeRate       1   13652871886   49682882313  943
## - DistanceToCity  1   98186590932  134216601360  988
## - SquareFootage   1 1511145948958 1547175959385 1098
```

Starting from the intercept-only model, this search can add variables and later remove variables it has already added.

What did R keep?

```
formula(forward_model)
```

```
## Price ~ SquareFootage + DistanceToCity + CrimeRate
```

```
formula(backward_model)
```

```
## Price ~ SquareFootage + CrimeRate + DistanceToCity
```

```r
formula(both_model)
```

```
## Price ~ SquareFootage + DistanceToCity + CrimeRate
```

For these data and this seed, all three methods keep square footage, crime rate, and distance, and exclude `RandomDummy`. The order of variables in the printed formulas may differ without changing the fitted model.

Compare the model with and without the random label:

```r
data.frame(
  Model = c("Housing predictors only", "Housing predictors plus dummy"),
  R_squared = c(
    summary(multi_model)$r.squared,
    summary(full_model)$r.squared
  ),
  AIC = c(AIC(multi_model), AIC(full_model))
)
```

```
##                            Model R_squared  AIC
## 1       Housing predictors only    0.9784 1060
## 2 Housing predictors plus dummy    0.9792 1061
```

The random dummy improves R^2 very slightly, but raises AIC. The small improvement in fit does not justify another coefficient under this rule.

```r
summary(backward_model)
```

```
##
## Call:
## lm(formula = Price ~ SquareFootage + CrimeRate + DistanceToCity,
##     data = houses_multi)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -71071 -21415  -2676  28029  54107
##
## Coefficients:
##                Estimate Std. Error t value             Pr(>|t|)
## (Intercept)    135813.9    18525.0    7.33     0.00000000563447 ***
## SquareFootage     186.6        4.5   41.47 < 0.0000000000000002 ***
## CrimeRate       -8127.5     2062.0   -3.94              0.00031 ***
## DistanceToCity  -5659.9      535.5  -10.57     0.00000000000028 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 29600 on 41 degrees of freedom
## Multiple R-squared:  0.978,  Adjusted R-squared:  0.977
## F-statistic:  620 on 3 and 41 DF,  p-value: <0.0000000000000002
```

This is one demonstration, not a guarantee that selection always removes irrelevant variables. Changing the random seed or the sample can change the result. We fixed the seed to keep the classroom example consistent.

4. Final Recap

- `lm()` fits a regression by minimizing squared residuals.
- `summary()` reports coefficients and measures of fit.
- `predict()` applies the fitted equation to new inputs.
- `step()` searches for a model that balances fit and complexity.

Keep three limits in mind: a coefficient does not establish causation; a high R^2 does not guarantee accurate predictions on new data; and predictions far outside the observed range can be unreliable.

Stepwise methods can also choose different models and do not guarantee the best model among every possible combination. Use the selected model as a starting point, with subject knowledge and checks on new data guiding its use.